Navigation Multi-Vues avec React Router

Objectifs

1. Installer et initialiser `react-router-dom` au sommet de l'application React.
2. Concevoir un composant de mise en page partagé (*Layout*) utilisant `<Outlet />` et `<NavLink>` pour la barre de navigation.
3. Structurer une table de routage déclarative avec `<Routes>` et `<Route>`.
4. Créer une vue détaillée dynamique exploitant `useParams` pour afficher une station spécifique depuis le Store Redux.
5. Déclencher une redirection programmatique avec `useNavigate` après la suppression d'une station.
6. Gérer les URLs inexistantes via une route générique `404` (`path="*"`).

Contexte du Projet : OmniMonitor - Architecture Multi-Pages

L'application **OmniMonitor** s'enrichit : au lieu de tout afficher sur un seul écran encombré, les opérateurs souhaitent disposer de vues dédiées (un tableau de bord général, une fiche technique détaillée par station météo et un centre d'alertes réseau), tout en conservant la rapidité et l'état Redux d'une Single Page Application.

Étape 1 : Installation et Configuration de `<BrowserRouter>`

1. Ouvrez un terminal à la racine de votre projet `omnimonitor-app`.
2. Installez la bibliothèque officielle de routage :

```shell
npm install react-router-dom
```

3. Ouvrez le fichier `src/main.jsx`.
4. Importez `BrowserRouter` depuis `'react-router-dom'` et enveloppez votre composant `<App />` à l'intérieur de `<BrowserRouter>` (tout en gardant `<Provider store={store}>` autour).

JavaScript

```
// src/main.jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { Provider } from 'react-redux';
import { BrowserRouter } from 'react-router-dom';
import { store } from './store/store.js';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </Provider>
  </StrictMode>
);
```

Étape 2 : Création de la Barre de Navigation et du Layout Partagé

1. Créez un composant `src/components/Navbar.jsx` :
    - Importez `NavLink` depuis `'react-router-dom'`.
    - Créez deux liens de navigation : l'un pointant vers `"/"` (libellé `"Tableau de bord"`) et l'autre vers `"/alerts"` (libellé `"Centre d'alertes"`).
    - Utilisez la fonction `className={({ isActive }) => ...}` pour appliquer un style visuel distinctif au lien actif.
2. Créez un composant `src/components/MainLayout.jsx` :
    - Importez `Outlet` depuis `'react-router-dom'`.
    - Affichez le composant `<Header />` et votre nouvelle `<Navbar />` en haut de la structure.
    - Placez la balise `<Outlet />` à l'intérieur d'une balise `<main>` pour accueillir le contenu des pages enfants.

Étape 3 : Création des Pages de l'Application

Créez un dossier `src/pages/` pour séparer clairement vos vues principales de vos petits composants réutilisables :

1. **`src/pages/DashboardPage.jsx` :** Déplacez-y la logique d'affichage de la barre de filtres (`<FilterBar />`) et de la grille des cartes de stations (`<StationCard />`) qui se trouvait auparavant dans `App.jsx`.
2. **`src/pages/AlertsPage.jsx` :** Créez une vue qui extrait depuis Redux uniquement les stations dont le statut est `'offline'` (`useSelector`) et les affiche sous forme de liste d'incidents prioritaires.
3. **`src/pages/NotFoundPage.jsx` :** Créez une page d'erreur `404` conviviale indiquant que la ressource demandée n'existe pas, accompagnée d'un composant `<Link to="/">` permettant de revenir au tableau de bord.

Étape 4 : Route Dynamique et Fiche Technique (`useParams` + `useNavigate`)

1. Créez la page `src/pages/StationDetailPage.jsx`.
2. Importez `useParams`, `useNavigate` et `Link` depuis `'react-router-dom'`, ainsi que `useSelector` et `useDispatch` depuis `'react-redux'`.
3. Récupérez le paramètre `stationId` depuis l'URL grâce à `const { stationId } = useParams();`.
4. Recherchez la station correspondante dans le Store Redux :

```javascript
const station = useSelector(state =>
  state.stations.list.find(st => st.id === Number(stationId))
);
```

5. Si la station n'existe pas, affichez un message d'erreur et un lien de retour vers `"/"`.
6. Si la station existe, affichez l'ensemble de ses informations (ID, nom, altitude, statut, nombre de capteurs).
7. Ajoutez un bouton `"Supprimer cette station et quitter"` qui :
    - Expédie l'action `dispatch(removeStation(station.id))`.

- Redirige immédiatement l'opérateur vers l'accueil via `navigate('/')`.

8. Dans `src/components/StationCard.jsx`, ajoutez un composant `<Link to={'/stations/' + id}>` intitulé `"Voir la fiche complète"` pour permettre d'accéder à cette vue.

Étape 5 : Assemblage de la Table de Routage dans `App.jsx`

Ouvrez `src/App.jsx` et remplacez son contenu par la déclaration des routes imbriquées :

JavaScript

```
// src/App.jsx
import { Routes, Route, Navigate } from 'react-router-dom';
import MainLayout from './components/MainLayout';
import DashboardPage from './pages/DashboardPage';
import StationDetailPage from './pages/StationDetailPage';
import AlertsPage from './pages/AlertsPage';
import NotFoundPage from './pages/NotFoundPage';

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<MainLayout />}>
        <Route index element={<DashboardPage />} />
        <Route path="stations/:stationId" element={<StationDetailPage />} />
        <Route path="alerts" element={<AlertsPage />} />
        <Route path="home" element={<Navigate to="/" replace />} />
        <Route path="*" element={<NotFoundPage />} />
      </Route>
    </Routes>
  );
}
```

Critères de Validation

Avant de valider ce segment, vérifiez que :

- La navigation entre `"/"`, `"/alerts"` et `"/stations/1"` est instantanée et ne déclenche aucun rechargement du navigateur.
- Le changement de statut d'une station sur le tableau de bord se répercute immédiatement dans l'onglet `"Centre d'alertes"` sans perte de l'état Redux.

- La suppression d'une station depuis sa fiche détaillée ramène automatiquement l'utilisateur sur `"/"`.
- La saisie d'une URL invalide (ex. `http://localhost:5173/test-inconnu`) affiche bien la vue `NotFoundPage` tout en conservant l'en-tête du `MainLayout`.